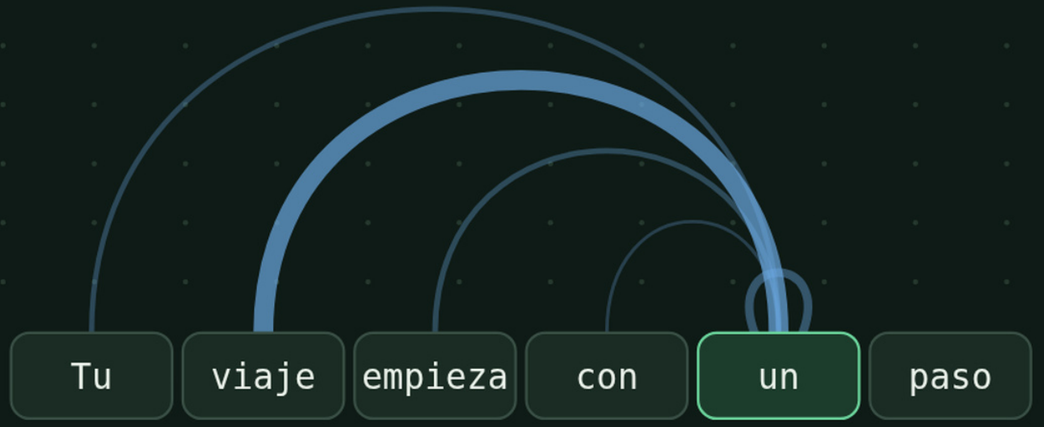
TALLER · PARTE 1 DE 2

From Tokens to Circuits

De los tokens a un GPT: tokenización, atención y la arquitectura completa

Rubén Fernández Boullón · Universidade de Vigo



EL MAPA

Las tres etapas para construir un LLM

ETAPA 1 · HOY

Construir

1. Preparar los datos
2. Mecanismo de atención
3. Arquitectura GPT

Capítulos 2, 3 y 4

ETAPA 2 · NOTEBOOK

Preentrenar

1. Bucle de entrenamiento
2. Evaluar la pérdida
3. Generar texto

Un run corto con Shakespeare

ETAPA 3 · OTRO DÍA

Ajustar

1. Clasificación
2. Seguir instrucciones

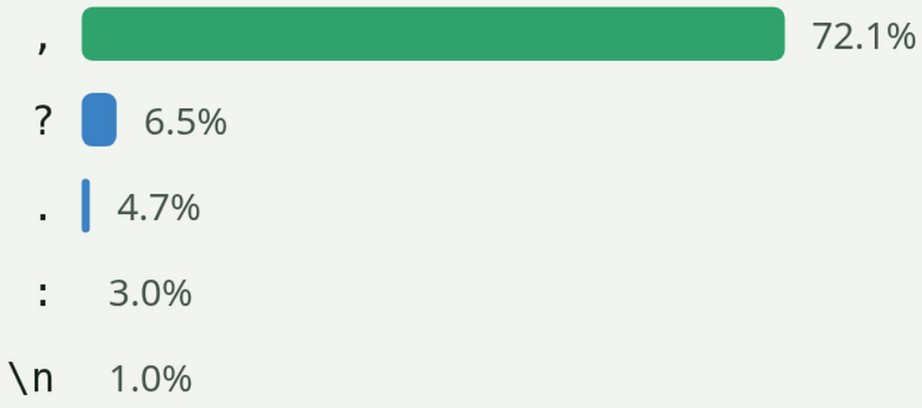
Fine-tuning

Según el esquema de S. Raschka, *Build a Large Language Model (From Scratch)*

LA IDEA CENTRAL

Un LLM solo hace una cosa: predecir el siguiente token

`To` `be` `,` `or` `not` `to` `be`



GPT-2 small, probabilidades reales.
El modelo puntúa los ~50.000 tokens del vocabulario, elegimos uno, lo añadimos y volvemos a empezar.

CAPÍTULO 2 · TEXTO → TOKENS

El modelo no lee letras: lee números

Inglés · tokenizador BPE de GPT-2

To	␣be	,	␣or	␣not	␣to	␣be	:	␣that	␣is	␣the	␣question	.
2514	307	11	393	407	284	307	25	326	318	262	1808	13

42 caracteres → 13 tokens

␣ = espacio. Ejemplos del notebook de la parte 1

BPE: fusionar una y otra vez el par más frecuente

l o w × 5

l o w e r × 2

n e w est × 6

w i d est × 3

Par más frecuente: «l» + «o» aparece 7 veces

Tokens nuevos del vocabulario

es est

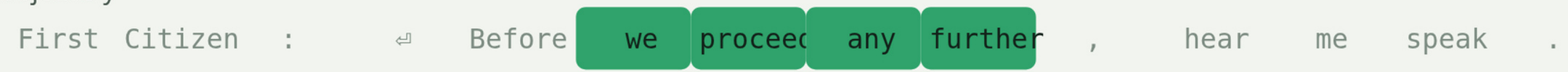
Corpus de juguete: palabra × frecuencia. GPT-2 repite esto hasta tener 50.257 tokens

El objetivo es la entrada desplazada una posición

entrada x



objetivo y



Before → we
Before we → proceed

context_length = 4: una ventana son 4 ejemplos de entrenamiento a la vez

Un id no dice nada; un vector, mucho

Ids o one-hot

Cada palabra es un 1 en su columna de un vector de 50.257 ceros.

perro	**1**	0	0	0	0	0	0	…
gato	0	**1**	0	0	0	0	0	…
Madrid	0	0	**1**	0	0	0	0	…
París	0	0	0	**1**	0	0	0	…
correr	0	0	0	0	**1**	0	0	…

«perro» está igual de lejos de «gato» que de «Madrid». Ninguna noción de parecido, y vectores gigantes.

Embedding aprendido

después de entrenar

Un vector denso, por ejemplo de 768 números, que se ajusta al entrenar.

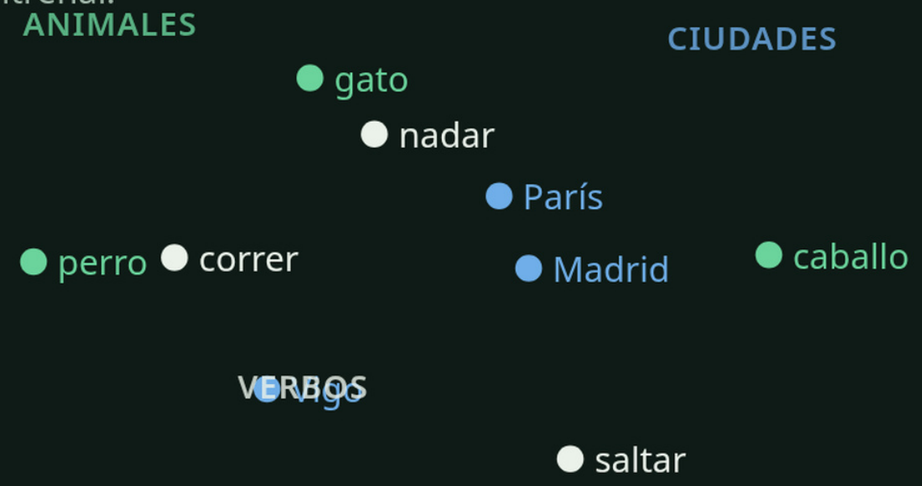


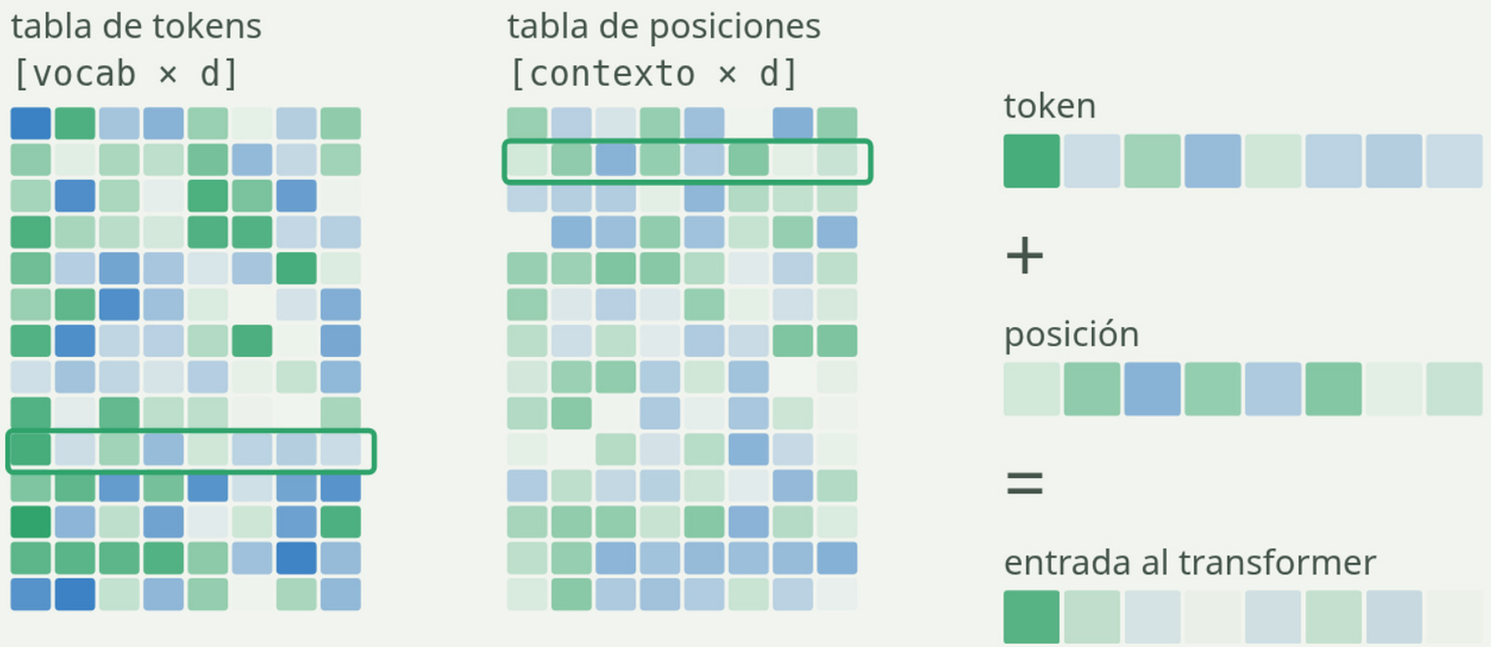
Ilustración en 2D. Los embeddings reales tienen cientos de dimensiones

CAPÍTULO 2 · EMBEDDINGS

Cada token es una fila de una tabla, más su posición

␣be

id 307 · pos 1



Valores ilustrativos. En GPT-2 small, d = 768; en nuestro mini-GPT, d = 128

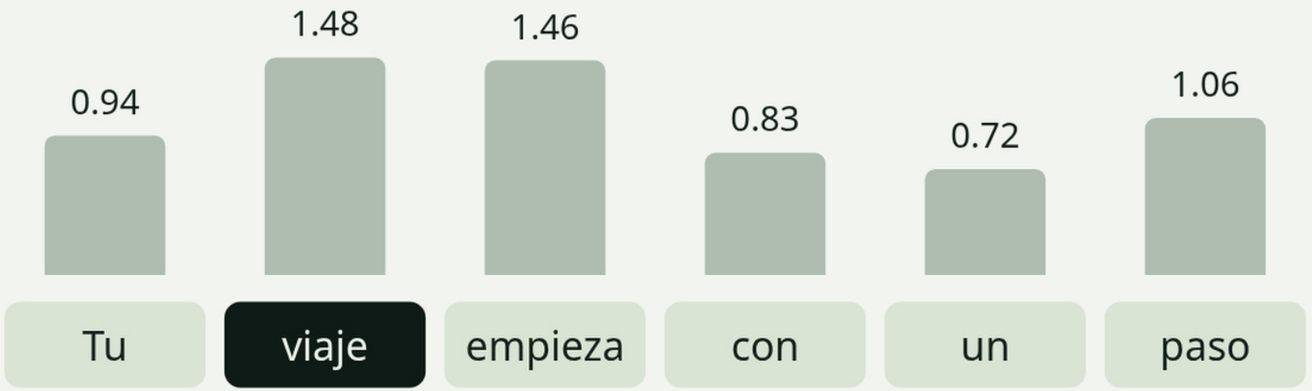
CAPÍTULO 3 · ATENCIÓN

Cada token decide a quién escuchar

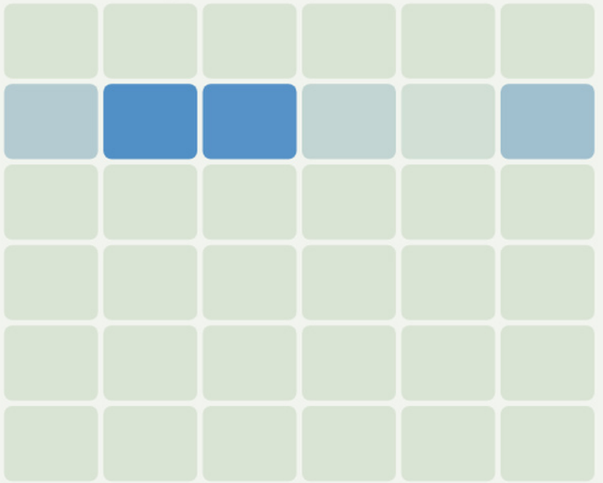
Y construye su nuevo vector como una **media ponderada** de los vectores de los demás.

CAPÍTULO 3 · ATENCIÓN SIMPLIFICADA

Similitud → softmax → media ponderada



Matriz de pesos de atención



1 · puntuación = producto escalar de «viaje» con cada token

Embeddings de juguete en 3 dimensiones, los mismos del notebook

CAPÍTULO 3 · UN PROBLEMA

Si atención = parecido, ¿no debería mirarme solo a mí?



Diagonal: cuánto se atiende cada token a sí mismo (15–24 %)

1 · No es total

Lo más parecido a mí soy yo, pero «viaje» solo se atiende un 24 %.

2 · Gana el vector grande

«con», «empieza» y «paso» miran más a «viaje»: el producto escalar premia la norma.

3 · Necesitamos parámetros

Un verbo busca a su sujeto, que no se le parece: hay que separar **qué busco**, **qué ofrezco** y **qué aporto**.

Mismos embeddings de juguete de la diapositiva anterior. Pesos = softmax($x \cdot x^T$)

Tres papeles para cada token: Query, Key, Value

$q = x \cdot W_Q$	**Query:** ¿qué estoy buscando?
$k = x \cdot W_K$	**Key:** ¿qué ofrezco para que me encuentren?
$v = x \cdot W_V$	**Value:** ¿qué información aporto si me eligen?

LA ANALOGÍA

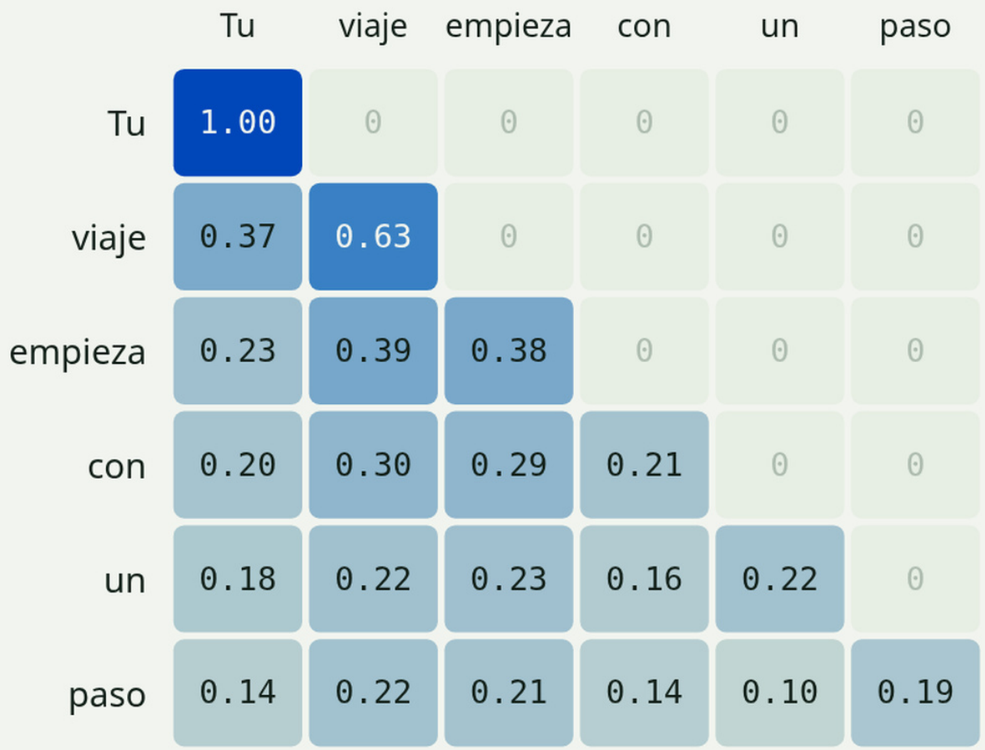
En una biblioteca, la **consulta** es lo que pides, las **claves** son las etiquetas del lomo y el **valor** es el contenido del libro.

$$\text{Attention}(Q,K,V) = \text{softmax}\left(Q \cdot K^T / \sqrt{d_k} \right) \cdot V$$

Dividir entre $\sqrt{d_k}$ evita que los productos crezcan con la dimensión y saturen la softmax

CAPÍTULO 3 · ATENCIÓN CAUSAL

Prohibido mirar al futuro



1 · Puntuaciones de todos con todos

2 · Por encima de la diagonal: −∞

3 · Softmax por filas: el futuro recibe peso 0

4 · Cada token solo ve el pasado y a sí mismo

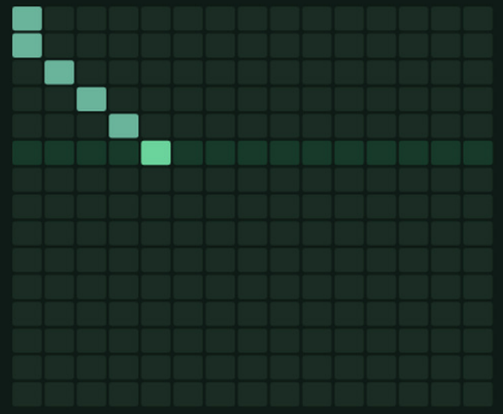
Filas: quién mira (consulta). Columnas: a quién mira (clave)

CAPÍTULO 3 · MULTI-HEAD ATTENTION

Varias cabezas en paralelo, cada una con su especialidad

consulta: **went**

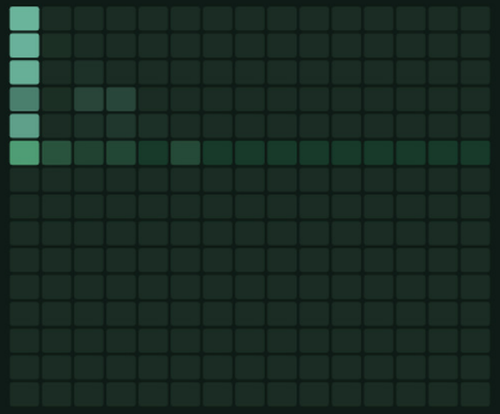
When Mary and John went to the store, John gave a drink to



L4H11

token anterior

→ **John** 100%



L4H4

sumidero: primer token

→ **⟨inicio⟩** 64%



L5H5

inducción: lo que vino después

→ **⟨inicio⟩** 97%



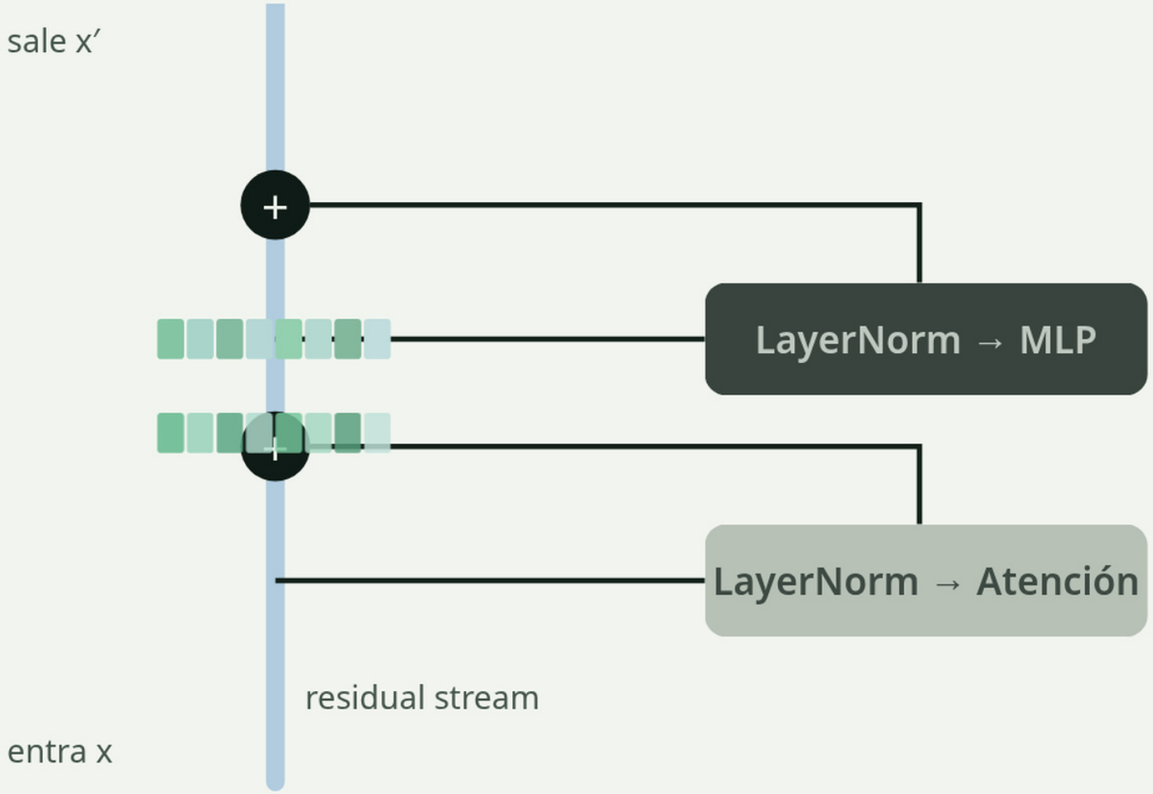
L9H9

mueve un nombre

→ **⟨inicio⟩** 100%

Patrones reales de 4 de las 144 cabezas de GPT-2 small (capa L, cabeza H)

Cada subcapa lee el vector y le suma algo




```
x = x + Atención(LN(x))
```

```
x = x + MLP(LN(x))
```

Atención mueve información *entre* tokens. El **MLP** procesa cada token por separado. Ninguna sobrescribe el vector: solo le **suman** su contribución.

Esquema pre-LayerNorm de GPT-2. Se repite N veces: 12 en GPT-2 small, 4 en nuestro mini-GPT

El GPT completo, de punta a punta



Tokens
ids enteros

Embeddings
token + posición

N bloques transformer
atención + MLP, sumando

Unembedding
vuelta al vocabulario

Logits
siguiente token: « be»

124 M
GPT-2 small
12 bloques · d = 768 · 12 cabezas

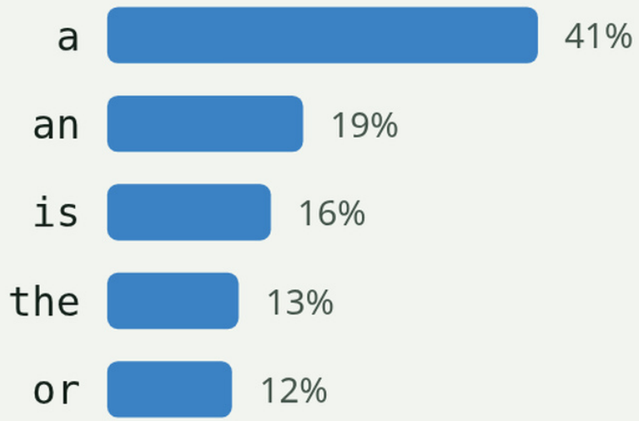
3,8 M
Nuestro mini-GPT
4 bloques · d = 128 · 4 cabezas

Misma arquitectura, distinta escala. Probabilidades de GPT-2 small tras «To be, or not to»

CAPÍTULO 4 · GENERAR TEXTO

La temperatura decide cuánto arriesga el modelo

Tras **“To be, or not to be,”** · top-5 de GPT-2, renormalizado



temperatura

2.0

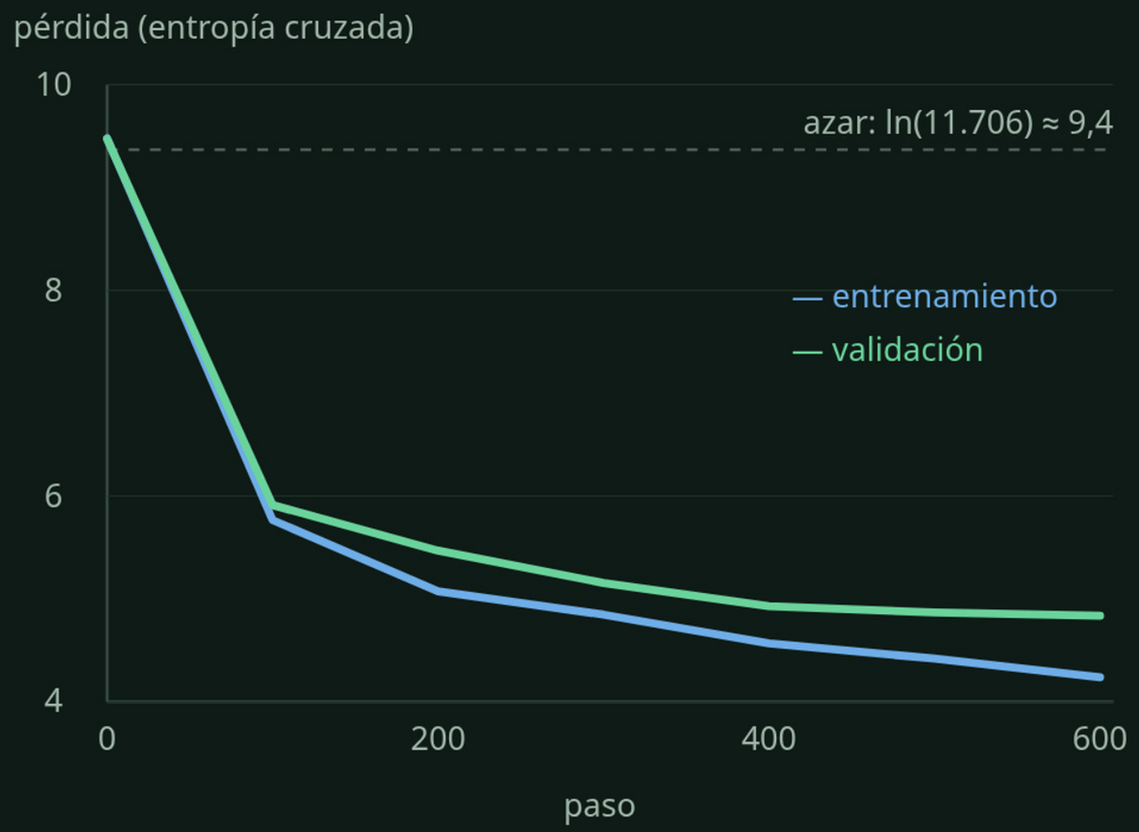
creativo / caótico

10 muestras: `an` `a` `the` `is` `an` `a` `the` `a` `a` `a`

Probabilidades reales de GPT-2 small, reescaladas con p^(1/T) dentro del top-5 (top-k = 5)

AL NOTEBOOK · PREENTRENAMIENTO EN MINIATURA

Seis minutos de Shakespeare en una CPU



paso 0 · pesos aleatorios

```
ROMEO:
suffering fliesglassere yourself captain
feelsSword bells Autates begspacedumedatt
```

paso 600

```
ROMEO:
Then, the duke;
Than of mine own king, and m
```

Run real del notebook: 600 pasos en CPU. En GPU (Colab T4), 2000 pasos en 1–2 minutos